

CSCI 450 – Artificial Intelligence
Neural Networks with Keras
100 pts (70 functionality, 30 commenting/format and analysis)

Overview:

In this assignment, you will be using Keras, an open source Python library that provides a common wrapper for several popular neural network libraries including Theano, TensorFlow and CNTK. We will be using TensorFlow as our back end. The following Python libraries have all been installed on the machines in the Mac and Linux labs and can be used in this assignment:

- Keras
- TensorFlow
- Scikit (including numpy)
- Sklearn
- Pandas

You will create your own neural network to identify four classes of breast tissue using the UCI breast tissue dataset.

For this lab, a venv (virtual environment for Python) has been provided on the Linux machines and vms.

```
source /usr/local/python-ai/bin/activate
```

To work in this environment, just use the command below and then run your python files as usual (for example `python breasttissue.py`). If you wish to create your own venv on your machine, send me an email and I'll send you a requirements.txt file that should hopefully help you install what you need.

Getting Started

I strongly recommend doing this tutorial to get started:

<https://machinelearningmastery.com/tutorial-first-neural-network-python-keras/>

Dataset

I've put a subset of a breast tissue dataset, called `breasttissue_train.csv`, in autolab as a handout. Download this csv from there. This is what you should use to train your neural net. I have also held out a test set that autolab will use to test your neural net to make predictions on unseen data.

The breast tissue dataset has 10 fields. The first is the class label – this is what you should be trying to learn/predict using the other 9 fields. The 9 fields represent impedance measurements of breast tissue samples at different frequencies. There are four classes that you are trying to identify: carcinoma, fad (a grouping of fibro-adenoma, glandular or mastopathy), connective, and adipose

Note that you are not allowed to modify the training file directly. You may only do so programmatically. Autolab will be loading the original training dataset so any changes that you make directly to the file will be lost in testing.

Technical Specifications

Input file:

You must read the input file as “`breasttissue_train.csv`” with no path info (it should be

stored in your working directory as you're coding, since this is where autolab will read it from).

Required function:

You will be implementing a Python function called `get_model` (with no parameters) that will return your compiled and fitted Keras *model* object, ready for testing. The function header should be:

```
def get_model():
```

Network structure:

You may use any number of hidden layers of any type (though I'd recommend sticking with Dense) with any activation method and with any number of neurons that you wish. However, your output layer must be a softmax layer. Softmax layers have a neuron for each possible output category and the activations of the neurons sum to 1. Each neuron's activation level can be viewed as the probability that the input corresponds to that particular class.

This blog entry might help you to better understand the softmax layer, as well as some of the manipulation you will need to do to the data file:

<https://machinelearningmastery.com/multi-class-classification-tutorial-keras-deep-learning-library/> A note on getting this to compile in our venv. The `np_utils` and `keras.scikitlearn` have been deprecated. So instead of the commented out lines below, you should import `to_categorical` and then use that object in the code in place of `np_utils` and import `KerasClassifier` from `scikeras.wrappers`.

```
#from keras.utils import np_utils
from keras.utils import to_categorical
# from keras.wrappers.scikit_learn import KerasClassifier
from scikeras.wrappers import KerasClassifier
```

Also note that you do NOT want to use the `KerasClassifier` method in your final lab code– this method will return a `SciKeras` object rather than a Keras *model* object. Stick with the `model.compile` and `model.fit` methods from the tutorial example. Here is the url for the Keras model object API: <https://keras.io/models/model/>

Exploration:

Experiment with this lab. You should obviously play with network structures but you should also experiment with training and testing. The `model.evaluate` method uses the input data and calculates loss and accuracy and any other metrics that you compiled. This is different than the `model.predict` method which will actually take inputs and predict the categories for those inputs. You can also experiment with validation sets or you can code your own k-fold cross-validation if you are curious about that.

Performance:

Anything that predicts 90% or higher of the held-out test set in autolab will be awarded full credit for performance. However, there will be some extra credit points offered for the highest scoring team. You can see the performance scores of submissions on the autolab scoreboard.

Commenting:

Comment your code and format consistently and well! It will be part of the overall grade.

Analysis:

Include a description of your exploratory process with the network structure, as well as your findings. This should be included as comments in your code file. This will be part of your overall grade.

Hints:

- The category labels in the file start at 1 rather than 0. Your program should account for this in your preprocessing of the data. You will probably want to use a one-hot encoded binary matrix for the class values (you can see this in the iris tutorial). Be sure that you don't change the relative order of the data categories (a category of 1 in the data file should correspond to a prediction of class 0, etc.)
- If you are consistently getting low performance results, have you scaled your data? In many machine learning techniques, and neural nets are no exception, values that have large ranges can overwhelm those that have small ranges (one column might have values from -1,000 to 1,000 and another column might range from 0 to 1, for example). Sklearn's *MinMaxScaler* can help with this.

Submission

Submit your .py file containing the `get_model()` function to autolab.